GitHub Actions + Azure: OIDC do zero

Manual prático para Cledson | Preparação GH-200 | 11/10/2026

Objetivo: criar uma identidade no Microsoft Entra ID, configurar a federação com um repositório GitHub, conceder leitura na assinatura e executar az group list sem client secret permanente.

O cenário deste laboratório

Item	Exemplo de demonstração
Aplicativo Entra	app-github-gh200
Proprietário GitHub	cledson-exemplo (troque pelo seu usuário ou organização)
Repositório	estudo-gh200
Branch autorizada	main
Papel Azure	Reader / Leitor na assinatura de estudo
Credencial federada	github-main
Workflow	.github/workflows/consultar-azure.yml

As telas oficiais mostram nomes de exemplo da Microsoft. As tabelas de preenchimento usam nosso cenário fictício. Nenhuma imagem representa sua conta. A aparência e os rótulos podem variar conforme idioma e atualizações do portal.

Roteiro

1. Preparar acessos e repositório.
2. Registrar o aplicativo e copiar IDs.
3. Atribuir Reader à identidade.
4. Conferir o subject OIDC.
5. Criar a credencial federada.
6. Cadastrar secrets.
7. Criar e executar o workflow.
8. Validar e diagnosticar.

Aqui você cria um **workflow** que consome a action azure/login. Não precisa escrever uma custom action nem manipular manualmente o JWT no fluxo normal.

1. Pré-requisitos e preparação

Tenha uma assinatura Azure ativa e um tenant Entra de trabalho. Confirme que o diretório do aplicativo será o da assinatura selecionada. No portal, o seletor de diretórios ajuda quando sua conta participa de vários tenants. [1]

Você precisa conseguir registrar aplicativos no tenant. Se o botão estiver indisponível, peça ao administrador o acesso apropriado. Para atribuir Reader, precisa de Microsoft.Authorization/roleAssignments/write no escopo, por exemplo via RBAC Administrator ou User Access Administrator. Ser Contributor não concede, por si só, esse poder. [1, 3]

No GitHub, crie ou escolha estudo-gh200, com branch padrão main e Actions permitido. Tenha acesso para alterar arquivos e cadastrar repository secrets. Use uma assinatura de estudo na qual leitura ampla seja adequada. [4]

Mapa das três configurações

Local	Função
Entra: aplicativo + credencial federada	Identifica o aplicativo e aceita o JWT de um contexto GitHub específico.
Azure: assinatura → IAM	Autoriza a identidade a ler recursos.
GitHub: secrets + workflow	Seleciona os IDs e solicita a autenticação.

O client ID aponta para a aplicação. O service principal é a representação dessa aplicação no tenant e recebe o papel Azure. O nome github-main é apenas um identificador da regra; não será enviado no YAML.

Condição de sucesso

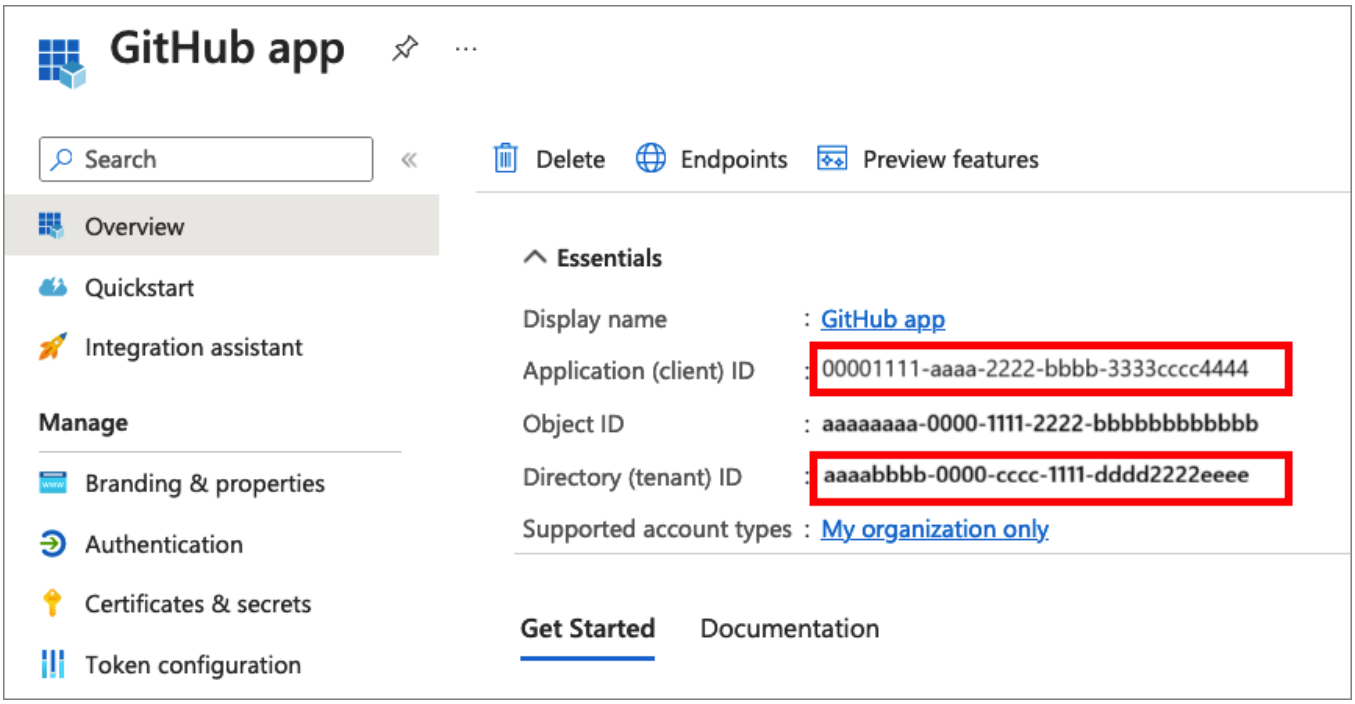
Você conhece seu tenant, assinatura, proprietário do repositório e branch; consegue editar o repositório e acessar os menus Azure necessários.

2. Registrar o aplicativo no Entra

Abra portal.azure.com → Microsoft Entra ID → Registros de aplicativo (App registrations) → Novo registro (New registration). Também é possível usar entra.microsoft.com. [1]

Campo	Preenchimento deste laboratório
Nome	app-github-gh200
Tipos de conta	Somente contas neste diretório organizacional (single tenant)
URI de redirecionamento	Deixe vazio para este fluxo de workload OIDC.
Ação final	Registrar / Register

Este quadro é um preenchimento ilustrativo, não uma captura do portal. Não crie client secret: o login será por credencial federada. Após registrar, abra Visão geral (Overview).



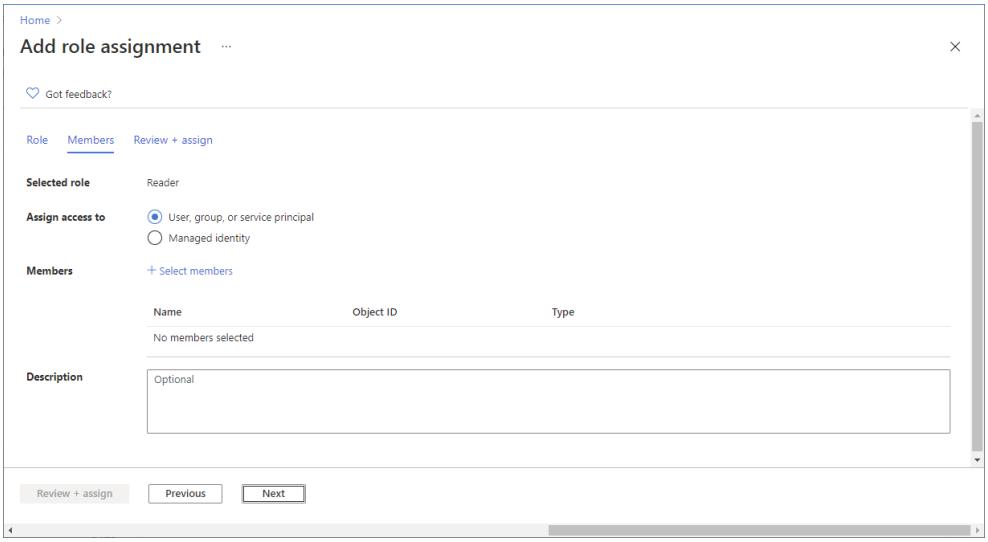
Tela oficial Microsoft: localização do Application (client) ID e Directory (tenant) ID. [2]

Anote: Application (client) ID e Directory (tenant) ID. O Object ID do aplicativo é diferente e será necessário somente se você criar a credencial via CLI. Não use Object ID no input client-id. [2]

Obtenha Subscription ID em Assinaturas → sua assinatura → Visão geral. Todos esses valores são identificadores, não senhas. Guarde-os para a etapa de secrets.

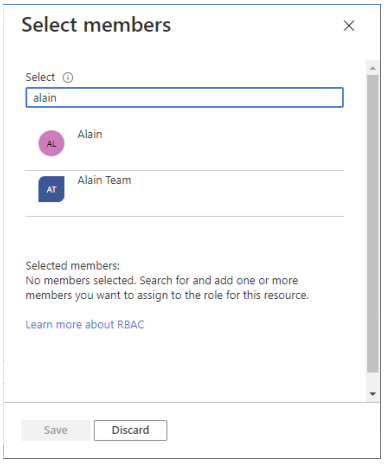
3. Dar permissão Reader na assinatura

Abra Assinaturas → sua assinatura de estudo → Controle de acesso (IAM) → Adicionar → Adicionar atribuição de função. Na guia Função, pesquise Reader / Leitor e selecione. Avance. [3]



Tela oficial Microsoft: guia Members. Selecione User, group, or service principal. [3]

Clique em Selecionar membros. Busque app-github-gh200 e escolha o service principal da aplicação correta. Se houver nomes repetidos, confira o identificador da aplicação. Não selecione sua própria conta pessoal por engano.



Tela oficial Microsoft: painel de busca de membros; os nomes mostrados são exemplos. [3]

Clique em Selecionar → Avançar → Revisar + atribuir. Se aparecer Assignment type, aplicações usam atribuição Active, não Eligible. Confira: papel Reader, membro app-github-gh200, escopo da assinatura escolhida. [3]

Reader permite leitura do gerenciamento dos recursos; não autoriza criar ou alterar recursos e não concede automaticamente leitura de dados internos, como conteúdo de blobs. [8]

4. Conferir o contexto e o subject OIDC

A credencial deve aceitar o subject real do job. Neste laboratório o disparo é manual em main e não há environment. Um job em pull_request ou com environment pode emitir outro contexto. [5]

Formato	Exemplo fictício
Anterior, baseado em nomes	repo:cledson-exemplo/estudo-gh200:ref:refs/heads/main
Imutável, com IDs	repo:cledson-exemplo@123/estudo-gh200@456:ref:refs/heads/main

Na referência atual, novos repositórios e renomeações/transferências a partir de 15/07/2026 usam o formato imutável. Repositórios existentes podem conservar o anterior ou aderir ao novo. Personalizações também alteram sub. Nunca use os IDs 123 e 456 literalmente. [6]

Opção para iniciantes: observar o sub antes do login

A próxima página oferece um workflow temporário que solicita um JWT com audience do Azure e exibe somente iss, aud e sub. Ele ainda não tenta entrar no Azure, portanto funciona antes de você cadastrar a confiança. Não precisa dos três secrets.

Crie o arquivo indicado na branch padrão. Em Actions, execute o workflow na branch main. Copie o subject mostrado para a credencial. Não mude branch ou acrescente environment depois sem ajustar a confiança.

Decodificar o payload serve para diagnóstico. Não valida a assinatura. A validação de confiança será feita pelo Entra durante a autenticação.

5. Opcional: solicitar JWT para diagnóstico

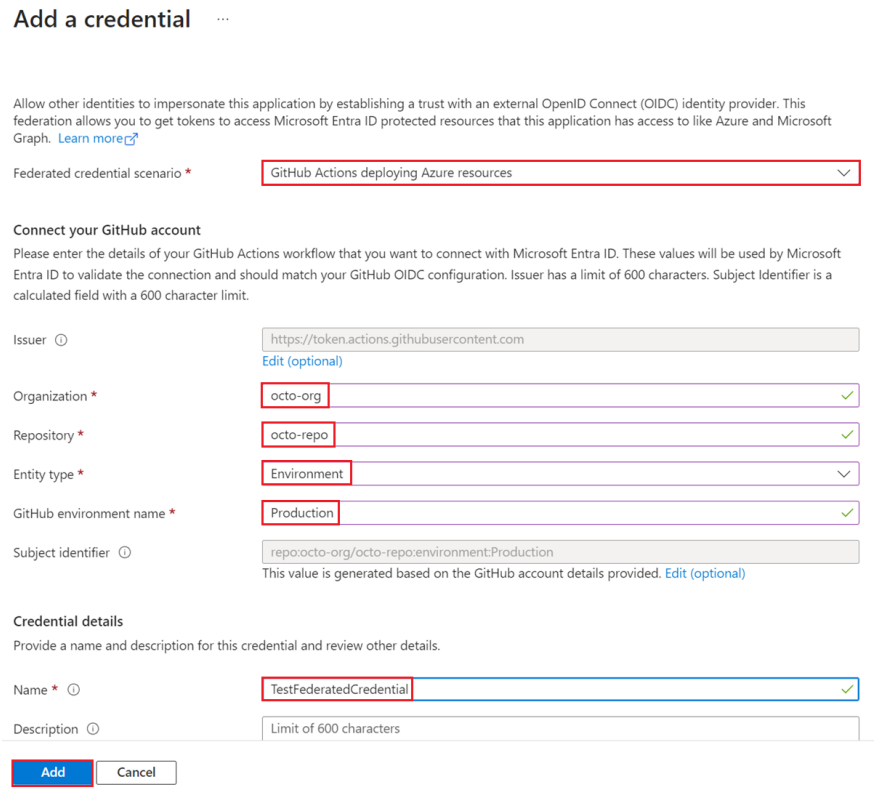
Crie .github/workflows/verificar-claims.yml. A permissão permite pedir o token; não concede acesso ao Azure. O script usa o endpoint e a autorização temporária disponibilizados ao job pelo runner. [5]

```
name: Conferir claims OIDC
on:
  workflow_dispatch:
jobs:
  claims:
    runs-on: ubuntu-latest
    permissions:
      id-token: write
    steps:
      - name: Exibir somente claims de configuração
        shell: bash
        run: |
          python3 - <<'PYCODE'
          import os, json, base64, urllib.request
          from urllib.parse import urlencode
          endpoint = os.environ["ACTIONS_ID_TOKEN_REQUEST_URL"]
          separator = "&" if "?" in endpoint else "?"
          endpoint += separator + urlencode({
              "audience": "api://AzureADTokenExchange"
          })
          request = urllib.request.Request(endpoint, headers={
              "Authorization": "Bearer " +
                  os.environ["ACTIONS_ID_TOKEN_REQUEST_TOKEN"]
          })
          with urllib.request.urlopen(request, timeout=30) as response:
              token = json.load(response)["value"]
          print("::add-mask::" + token)
          part = token.split(".")[1]
          payload = json.loads(base64.urlsafe_b64decode(
              part + "=" * (-len(part) % 4)))
          for key in ("iss", "aud", "sub"):
              print(key + ": " + str(payload[key]))
          PYCODE
```

Os logs terão issuer do GitHub, audience api://AzureADTokenExchange e o subject do seu job. O JWT completo e o token de requisição não são impressos nem salvos em outputs ou artifacts. Remova o workflow temporário quando terminar.

6. Criar a credencial federada

Volte ao app-github-gh200 → Certificados e segredos → Credenciais federadas → Adicionar credencial. Para o formato anterior, selecione o cenário GitHub Actions deploying Azure resources e preencha proprietário, repositório e Branch main. [2]



Tela oficial Microsoft de exemplo. Use seus valores, não os nomes desta captura. [2]

Campo	Valor esperado
Nome	github-main
Issuer	https://token.actions.githubusercontent.com
Audience	api://AzureADTokenExchange
Subject	O sub observado na etapa anterior.

Se o assistente gerar um subject diferente do observado, use Other issuer e Explicit subject identifier, se disponíveis, para informar o valor exato. Confira maiúsculas, branch, nomes e IDs; então salve. Não cadastre um wildcard amplo para contornar erros. [2, 6]

7. Alternativa: cadastro pelo Azure Cloud Shell

Use esta alternativa se o portal não permitir editar o subject correto. Não execute as duas alternativas para criar regras duplicadas. No Cloud Shell, selecione Bash e confirme o tenant correto. [6]

Crie credential.json com o editor, substituindo o subject pelo valor exato observado. O issuer não tem barra final neste exemplo.

```
{
  "name": "github-main",
  "issuer": "https://token.actions.githubusercontent.com",
  "subject": "SUBJECT_EXATO_OBSERVADO",
  "audiences": ["api://AzureADTokenExchange"]
}
```

Copie o Object ID de app-github-gh200, na Visão geral do registro. Execute, substituindo o placeholder:

```
az ad app federated-credential create \
  --id "OBJECT_ID_DO_REGISTRO_DO_APLICATIVO" \
  --parameters ./credential.json
```

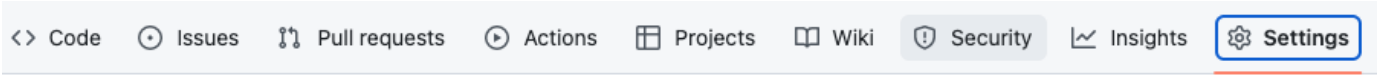
Essa operação administrativa configura o aplicativo. Ela exige autorização para editar o registro no Entra; o Reader da assinatura não concede essa autorização. Confira a nova regra na guia Credenciais federadas.

Antes de prosseguir

A regra está no mesmo aplicativo cujo Client ID será usado no GitHub. Issuer, audience e subject coincidem com os claims. Reader foi atribuído ao service principal na assinatura. Não foi criado client secret.

8. Cadastrar os secrets no GitHub

Abra estudo-gh200 → Settings → Secrets and variables → Actions → Secrets → New repository secret. Cadastre cada nome e seu valor real, depois clique em Add secret. [4]



Tela oficial Microsoft: menu de secrets no GitHub. [7]

Name	Valor a colar em Secret
AZURE_CLIENT_ID	Application (client) ID de app-github-gh200
AZURE_TENANT_ID	Directory (tenant) ID do mesmo aplicativo
AZURE_SUBSCRIPTION_ID	Subscription ID da assinatura com Reader

Exemplo de formulário ilustrativo: Name = AZURE_CLIENT_ID; Secret = seu GUID real, sem aspas e sem espaços extras. Repita para os outros dois. A lista final deve conter os três nomes.

O GitHub não mostra novamente o valor salvo; se estiver errado, use Update para substituí-lo. Um secret inexistente resulta em valor vazio na expressão. Não crie AZURE_CLIENT_SECRET neste cenário. [4]

Esses IDs não são senhas. Usamos secrets para centralizar a configuração e acompanhar o padrão do exemplo oficial de login. A federação continua sendo o mecanismo de autenticação.

9. Criar o workflow de consulta

Na aba Code, use Add file → Create new file. Informe .github/workflows/consultar-azure.yml, cole o conteúdo e faça commit em main. Se sua política exigir pull request, faça merge para a branch padrão antes do primeiro disparo manual. [9]

```
name: Consultar Azure com OIDC
on:
  workflow_dispatch: # Botão Run workflow.

jobs:
  consultar:
    runs-on: ubuntu-latest # Azure CLI disponível no runner.
    permissions:
      id-token: write # Autoriza solicitar o JWT.
    steps:
      - name: Login no Azure por federação
        uses: azure/login@v3 # Tag major do exemplo didático.
        with:
          # Identifica o aplicativo com a credencial federada.
          client-id: ${{ secrets.AZURE_CLIENT_ID }}
          # Identifica o diretório Microsoft Entra.
          tenant-id: ${{ secrets.AZURE_TENANT_ID }}
          # Seleciona a assinatura com Reader.
          subscription-id: ${{ secrets.AZURE_SUBSCRIPTION_ID }}
          # Corresponde à audience da federação no Azure público.
          audience: api://AzureADTokenExchange

      - name: Conferir assinatura selecionada
        run: az account show --query name --output tsv

      - name: Listar grupos de recursos
        run: |
          az group list \
            --query "[].{Nome:name,Localizacao:location}" \
            --output table
```

Não há checkout porque o job não lê arquivos locais; contents: read só seria necessário para operações que exigem essa permissão. Não há environment: o contexto da confiança é a branch main.

azure/login solicita o JWT, envia-o ao Entra e estabelece a sessão. A versão v3 é usada como referência didática; fixe o SHA completo de uma release revisada quando sua política exigir pinning. [10]

10. Executar e interpretar os resultados

Abra Actions → Consultar Azure com OIDC → Run workflow. Selecione main e confirme. Abra a execução e o job consultar; expanda cada step. O arquivo com workflow_dispatch precisa estar na branch padrão para o disparo manual. [9]

Step	O que confirmar
Login no Azure	Terminou com sucesso: o Entra aceitou a federação.
Conferir assinatura	Mostra o nome esperado para AZURE_SUBSCRIPTION_ID.
Listar grupos	Exibe grupos visíveis na assinatura selecionada.

```
# Saída ilustrativa; não é resultado da conta de Cledson.
Nome              Localizacao
----------------  -----------
rg-estudos        brazilsouth
rg-aplicacoes     eastus
```

Uma lista vazia não prova falha de OIDC. Pode não haver grupos no escopo selecionado; confira assinatura e permissões. Reader na assinatura é o escopo deste laboratório; em produção, avalie limitar ao recurso ou grupo necessário. [8]

O caminho percorrido

GitHub emite JWT → azure/login apresenta JWT ao Entra → Entra verifica a credencial federada no aplicativo → emite access token → Azure Resource Manager verifica RBAC → az group list consulta os grupos. [5, 10]

id-token: write habilita a primeira parte. A credencial federada permite a troca. Reader autoriza a leitura. Nenhuma dessas configurações substitui as outras.

11. Resolver problemas e concluir

Sintoma	Verificação prática
JWT não pode ser solicitado	Confira id-token: write nas permissões efetivas do job.
AADSTS700213 / nenhum registro federado	Compare sub, iss e aud com a regra, inclusive formato com IDs e branch main.
Login sem assinatura acessível	Confira subscription ID, tenant e atribuição RBAC ao service principal correto.
AuthorizationFailed	Confira papel, escopo e operação. Reader não faz deploy.
Botão Run workflow ausente	Confira workflow_dispatch e arquivo na branch padrão.
Input de login vazio	Confira os três repository secrets e os nomes no YAML.
Alteração recém-feita ainda falha	Aguarde propagação e execute novamente; não amplie a confiança sem diagnóstico.

Ao concluir, mantenha somente a regra necessária e remova o workflow temporário de claims. Se abandonar o laboratório, remova a atribuição Reader e a credencial federada de teste; exclua o aplicativo apenas se ele não tiver outros consumidores.

Critério de conclusão

A execução em main autentica sem client secret, seleciona a assinatura certa e consulta os grupos. Um contexto diferente não deve autenticar usando esta regra de main. Não transforme um teste de leitura em deploy antes de revisar os papéis necessários.

Este manual documenta um cenário reproduzível; não realizou alterações no Azure nem no GitHub da sua conta.

Referências oficiais e telas

[1] Registro de aplicativos

[2] Confiança federada e capturas Entra

[3] Atribuir papéis e capturas IAM

[4] Repository secrets

[5] OIDC: claims e permissão

[6] Subjects imutáveis no Entra

[7] Conectar GitHub e Azure; captura GitHub

[8] Papel Reader

[9] Disparar workflow manual

[10] Azure Login: documentação da action

Telas reproduzidas de documentação oficial da Microsoft; nomes nas capturas não são dados do usuário. Os quadros preenchidos e saídas marcados como ilustrativos foram criados para explicar este laboratório. Links clicáveis permitem consultar a versão atual.